EMAIL SPAM CLASSIFICATION USING NAIVE BAYES

DWDM - CSA1614

Name: Rohith Manikanta Rellu
Register Number: 192425326

Aim

To classify incoming emails as Spam or Not Spam using prior and conditional probabilities with the Naive Bayes method.

Software and Libraries Required

Python, Jupyter Notebook, and Pandas.

Tasks Performed

1. Create a dataset containing at least 10 sample emails with relevant attributes.
2. Calculate prior probabilities for the two classes.
3. Calculate conditional probabilities using the Naive Bayes method.
4. Classify a new email using the calculated probabilities.
5. Explain the role of Bayes' theorem in the classification process.

Task 1: Create a Dataset of Sample Emails

A dataset of 12 sample emails is created using two attributes: Offer and Link. The Class column is the target label, with Spam and Not Spam as the two classes.

Email_ID	Offer	Link	Class
1	Yes	Yes	Spam
2	Yes	Yes	Spam
3	Yes	No	Spam
4	Yes	Yes	Spam
5	Yes	No	Spam
6	No	No	Spam
7	No	No	Not Spam
8	No	Yes	Not Spam
9	No	No	Not Spam
10	No	Yes	Not Spam
11	No	No	Not Spam
12	No	No	Not Spam

Jupyter Notebook - Cell 1

```
import pandas as pd

data = {
    'Offer': ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No',
              'No', 'No', 'No', 'No', 'No', 'No'],
    'Link':  ['Yes', 'Yes', 'No', 'Yes', 'No', 'No',
              'No', 'Yes', 'No', 'Yes', 'No', 'No'],
    'Class': ['Spam', 'Spam', 'Spam', 'Spam', 'Spam', 'Spam',
              'Not Spam', 'Not Spam', 'Not Spam', 'Not Spam',
              'Not Spam', 'Not Spam']
}

df = pd.DataFrame(data)
df.index = df.index + 1
df.index.name = 'Email_ID'
df
```

The output contains 12 sample records, with 6 Spam and 6 Not Spam emails.

Jupyter Output 1: Sample Email Dataset

The screenshot below shows the dataset displayed in Jupyter Notebook.

Output of Cell 1: Dataset

[1]:

Email_ID	Offer	Link	Class
1	Yes	Yes	Spam
2	Yes	Yes	Spam
3	Yes	No	Spam
4	Yes	Yes	Spam
5	Yes	No	Spam
6	No	No	Spam
7	No	No	Not Spam
8	No	Yes	Not Spam
9	No	No	Not Spam
10	No	Yes	Not Spam
11	No	No	Not Spam
12	No	No	Not Spam

Task 2: Calculate Prior Probabilities

Prior probability is the probability of a class before considering email attributes. There are 6 Spam and 6 Not Spam emails out of 12 records.

P(Spam) = 6/12 = 0.5 P(Not Spam) = 6/12 = 0.5

Jupyter Notebook - Cell 2

```
class_counts = df['Class'].value_counts()
total = len(df)
priors = class_counts / total

print('Prior Probabilities:')
print(priors)
```

Output of Cell 2: Prior Probabilities

```
Prior Probabilities:
Class
Spam        0.5
Not Spam    0.5
Name: count, dtype: float64
```

Task 3: Calculate Conditional Probabilities

Conditional probability estimates how often an attribute value occurs within a class. For example, P(Offer = Yes | Spam) = 5/6 = 0.833.

Jupyter Notebook - Cell 3

```
for attribute in ['Offer', 'Link']:
    print(f'\nConditional probabilities for {attribute}:')
    print(pd.crosstab(
        df[attribute], df['Class'], normalize='columns'
    ).round(3))
```

Output of Cell 3: Conditional Probabilities

```
Conditional probabilities for Offer:
Class  Not Spam   Spam
Offer
No          1.0  0.167
Yes         0.0  0.833

Conditional probabilities for Link:
Class  Not Spam  Spam
Link
No        0.667   0.5
Yes       0.333   0.5
```

Task 4: Classify a New Email

Consider a new email with Offer = Yes and Link = Yes. Naive Bayes multiplies the prior probability by the conditional probabilities for each attribute.

$$\textbf{Spam score} = 0.5 \times (5/6) \times (3/6) = 0.2083$$

$$\textbf{Not Spam score} = 0.5 \times 0 \times (2/6) = 0$$

Jupyter Notebook - Cell 4

```
email = {'Offer': 'Yes', 'Link': 'Yes'}
scores = {}

for cls in df['Class'].unique():
    probability = priors[cls]
    for attribute, value in email.items():
        conditional = ((df[attribute] == value) &
                       (df['Class'] == cls)).sum() / (df['Class'] == cls).sum()
        probability *= conditional
    scores[cls] = probability

prediction = max(scores, key=scores.get)
print('New Email:', email)
print('Class Scores:', scores)
print('Predicted Class:', prediction)
```

Output of Cell 4: Email Classification

```
New Email: {'Offer': 'Yes', 'Link': 'Yes'}
Class Scores: {'Spam': np.float64(0.20833333333333334), 'Not Spam': np.float64(0.0)}
Predicted Class: Spam
```

Prediction: the new email is classified as Spam because its Spam score is greater than its Not Spam score.

Task 5: Role of Bayes' Theorem

Bayes' theorem updates the probability of a class after observing evidence. In email classification, it combines the prior class probability with the conditional probabilities of attributes such as Offer and Link. Naive Bayes assumes that these attributes are conditionally independent given the class.

$$\textbf{P(Class | Evidence)} \propto \textbf{P(Class)} \times \textbf{P(Evidence | Class)}$$

Result

A 12-record email dataset was created, prior and conditional probabilities were calculated, and a new email with Offer = Yes and Link = Yes was classified as Spam using the Naive Bayes method.

Note: This is a small educational example. Zero conditional probabilities can be avoided in practical applications by using Laplace smoothing.